DOCS · 11 ~25 MIN

Developer API

One key and one base URL. Cloud endpoints (chat, computer-use actions, images) answer from Twinly's servers. Mac endpoints (send and read iMessages, place a call in your cloned voice, speak a line in your voice, run a full task) run on the key owner's Mac, in their running Twinly app, under their own approval settings. Every call is charged to the account that owns the key, on its existing plan.

ON THIS PAGE

01 Start here
02 The basics
03 Which endpoint do I want?
04 What you can build
05 Get a key
06 Quickstart
07 Chat: POST /v1/chat/completions
08 Computer use: POST /v1/computer/next-action
09 Images: POST /v1/images
10 Mac endpoints: how jobs work
11 Send and read messages
12 Place a call: POST /v1/calls
13 Speak in the owner's voice: POST /v1/voice/speak
14 Run a task: POST /v1/tasks
15 Check or cancel a job
16 Webhooks and bots
17 Readiness and usage
18 Pricing and limits
19 Errors
20 Security

01 Start here

From a fresh Mac to a working call in about ten minutes:

Download Twinly from twinly.tech, move it to Applications and open it.

Sign in with Google when Twinly asks (or on the Account screen). Keys belong to the signed-in account.

Accept the Terms when Twinly shows them. Until you accept, Twinly runs nothing for the API: the Mac stops checking in, so every Mac endpoint answers `409 twinly_offline`. Cloud endpoints work either way.

Make a key: **Settings ▸ Developer API ▸ Create a key**. Copy it right away; it is shown once.

Let keys use this Mac. In the same section, the **Jobs on this Mac** card has the switch **Let keys text, call and run tasks as you**. It is on by default; keep it on. With it off, Mac endpoints answer `409 twinly_offline` and no webhook or bot event is sent.

Optional: redeem a code in **Settings ▸ Developer API ▸ Redeem a code** (also on Account). A promo code from Twinly adds usage to the account (each account can redeem a code once).

Keep Twinly open on the Mac, signed in, with the Mac awake, whenever your code calls a Mac endpoint (messages, calls, voice, tasks).

Call `GET /v1/me`. It is free and needs no scope, and it tells you whether your Mac is online and which endpoints this key can call right now. With Twinly 4.1.5 or later, `mac.setup` also says, per family of endpoints, whether the Mac is ready (`true`) or what to set up first (a sentence), for example `"calls": "Needs: install BlackHole 2ch (Twinly, Voice)."`; it is `null` from an older Twinly or while the Mac is offline.

BASH

```
curl -s https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1/me \
  -H "Authorization: Bearer $TWINLY_API_KEY"
```

Cloud endpoints (chat, images, computer next-action) need only steps 1 to 4. Each Mac endpoint also needs its own piece of the Mac set up once; see Set up the Mac.

02 The basics

Base URL	`https://advtwnedtghmswpxokg.supabase.co/functions/v1/twinly-api/v1`
Auth	`Authorization: Bearer twk_live_...`
Model	`twinly-core`
OpenAPI 3.1	/openapi.yaml, for Postman, Insomnia, or any client generator
Mac endpoints need	Twinly 4.1.2 or later (the current download is), open and signed in on the key owner's Mac, with the Terms accepted
Starter projects	github.com/OjasMishra32/twinly-api-examples: six projects with tests and a mock API, no installs

A key belongs to one Twinly account and acts as that person: texts come from their number, calls speak in their voice, tasks run on their Mac. Build with your own key on your own Mac.

Cloud endpoints answer in the request. **Mac endpoints** create a **job** the Mac picks up within seconds; you wait on it with `?wait=` or get a webhook when it is done.

One usage pool. API calls spend the same plan and extra usage as the app. `GET /v1/usage` shows where you are.

Scopes limit each key to what a project needs.

03 Which endpoint do I want?

I want to	Use	Answers
Ask a model a question, with or without images	`POST /v1/chat/completions`	now
Make a picture	`POST /v1/images`	now
Drive a screen my code controls, one step at a time	`POST /v1/computer/next-action`	now
Text someone (with pictures, audio or files) from my number	`POST /v1/messages`	job, about 2 s

Get my cloned voice saying something, as audio	`POST /v1/voice/speak`	job, about 3 s
Phone a business and get the transcript	`POST /v1/calls`	job, minutes
Have my Twinly do a whole thing (research, browser, calendar, files, the screen)	`POST /v1/tasks`	job, seconds to minutes
Get the answer back as JSON fields	`POST /v1/tasks` with `output_schema`	job
React when someone texts me, or build an @twinly bot	webhooks	push
Check my key and Mac before a demo	`GET /v1/me`	now, free

04 What you can build

Each endpoint is one power of the owner's twin. Apps get interesting when you chain them: read a chat, think about it, answer in the owner's voice, call someone, or hand the whole job to the Mac as a task. Everything below works with one key.

Combine	And you get	Endpoints
read a chat + chat + send	a digest, a summary or a reply drafted from what people actually said	`GET /v1/messages`, `POST /v1/chat/completions`, `POST /v1/messages`
bot command + your code + reply	an `@twinly <command>` bot that works in any iMessage chat, groups included	`bot.command` webhook, your reply (or `POST /v1/messages`)
incoming text + chat + send	an assistant that answers in the chats the owner picked	`message.received` webhook, `POST /v1/chat/completions`, `POST /v1/messages`
image + send	a picture made from a sentence and texted to someone	`POST /v1/images`, `POST /v1/messages` with an attachment

	voice	`/v1/messages` with the audio
call + text	a phone call to a business with a goal, then the outcome texted back	`POST /v1/calls` , `POST /v1/messages`
task + JSON	the Mac researches, browses, reads its calendar, mail or files, and returns fields your code can use	`POST /v1/tasks` with `output_schema`
task on a schedule + alert	a watcher: a price, a slot, a page that changes, then a text or a call when it does	your scheduler, `POST /v1/tasks` , `POST /v1/messages` or `POST /v1/calls`
screenshot + next action, in a loop	your own computer-use agent for a screen your code controls (a VM, a Playwright browser, a game)	`POST /v1/computer/next-action`

Project ideas

Group-chat concierge. `@twinly poll` , `@twinly weather` , `@twinly split 84 4` in a group chat; your webhook answers in the chat. (`bot.command`)

Morning digest. Every morning, read the busiest group chat and text the owner three lines: decisions, questions, plans. (`GET /v1/messages` , chat, `POST /v1/messages`)

The booking caller. Give it a restaurant's number and a goal ("a table for 4 at 7, somewhere quiet if they can"): the twin calls, you get the transcript, and the owner gets a text with the result. (`POST /v1/calls` , `POST /v1/messages`)

Slot sniper. Check a booking page every 10 minutes with a task that returns `{"open": true, "times": [...]}` , then text or call the owner the moment a slot opens. (`POST /v1/tasks` + `output_schema`)

Price watch. A task reads a product page and returns the price as JSON; text the owner when it drops. (`POST /v1/tasks`)

Voice postcards. Draw a picture from a sentence, add a voice note in the owner's voice, text both to a friend. (`POST /v1/images` , `POST /v1/voice/speak` , `POST /v1/messages`)

Follow-up nudger. Find questions people asked the owner that got no answer this week and list them, with drafts. (`GET /v1/messages` , chat)

Meeting prep. A task reads tomorrow's calendar and the related email threads and returns a one-page brief as JSON for your UI. (`POST /v1/tasks`)

Read it to me. Turn any article into a short audio summary in the owner's voice. (`POST /v1/tasks` or chat, `POST /v1/voice/speak`)

Text adventure or trivia in a group chat. Your server keeps the game state; `@twinly` commands play it. (`bot.command`)

Your own computer-use agent. Screenshot a VM or a headless browser, ask for the next action, perform it, repeat; benchmark it against a task list. (`POST /v1/computer/next-action`)

Wake-up call. At 7, the twin calls the owner and reads the day's calendar and the weather in their own voice. (`POST /v1/tasks` , `POST /v1/calls`)

Bill splitter. `@twinly split 84.50 4 tip 20` in a group chat: your webhook does the math and posts each share back. (`bot.command`)

Notes for coding agents

Call `GET /v1/me` first: it says whether the owner's Mac is online, what this key may call, and (Twinly 4.1.5+) what each family of endpoints still needs set up in `mac.setup` .

Mac endpoints are jobs that run on the owner's Mac under their approval settings. A held step shows `needs_approval` until they decide. A Mac runs up to two jobs at once, and one call or task at a time.

A task only texts, emails or calls someone when its prompt asks for that, and only the person it names. To send something yourself, use `POST /v1/messages` .

Calls dial US numbers through the owner's Google Voice and need the setup in Set up the Mac.

`POST /v1/computer/next-action` is for screens your code controls; it returns one action and never touches the owner's Mac.

Twinly never solves robot checks or types card numbers or Social Security numbers. Plan around both.

Test without a key or a Mac against the mock API in the starter projects; it answers like production.

if you like, set a monthly spending cap. The key is shown **once**: copy it into your server's environment or a password manager. Twinly keeps only a fingerprint and can never show it again; if you lose it, revoke it and make a new one.

No Mac, or building on Windows or Linux? Sign in with Google at twinly.tech/developers to make, list and revoke keys and redeem a promo code. Those keys work with every cloud endpoint (chat, images and computer use, where your own code takes the screenshots and clicks on any computer). Texts, calls and tasks still need Twinly running on a Mac.

BASH

```
export TWINLY_API_KEY=twk_live_...   # never commit this
export API=https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1
```

Scopes

A new key can do everything; untick what a project does not need in Settings ▸ Developer API. A request outside a key's scopes gets `403 scope_missing`, and the message names the scope.

Scope	Allows
`chat`	`POST /v1/chat/completions`
`computer`	`POST /v1/computer/next-action`
`images`	`POST /v1/images`
`messages.send`	`POST /v1/messages`
`messages.read`	`GET /v1/messages`
`calls`	`POST /v1/calls`
`voice`	`POST /v1/voice/speak`
`tasks`	`POST /v1/tasks`
`bots`	@twinly command webhooks (set up in the app)

`GET /v1/me`, `GET /v1/usage`, `GET /v1/models` and a key's own jobs need no scope.

The chat endpoint speaks OpenAI's format, so the official `openai` packages work by changing the base URL.

BASH

```
curl https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1/chat/completion
  -H "Authorization: Bearer $TWINLY_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "twinly-core", "messages": [{"role": "user", "content": "Write a haiku a
```

PYTHON

```
import os
from openai import OpenAI

client = OpenAI(
    base_url="https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1",
    api_key=os.environ["TWINLY_API_KEY"],
)
reply = client.chat.completions.create(
    model="twinly-core",
    messages=[{"role": "user", "content": "Write a haiku about Gainesville."}],
)
print(reply.choices[0].message.content)
```

JS

```
import OpenAI from "openai";

const client = new OpenAI({
  baseURL: "https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1",
  apiKey: process.env.TWINLY_API_KEY,
});
const reply = await client.chat.completions.create({
  model: "twinly-core",
  messages: [{ role: "user", content: "Write a haiku about Gainesville." }],
});
console.log(reply.choices[0].message.content);
```

> Call the API from a server or a script, never from a web page: the API refuses browser requests (`403 origin_not_allowed`), and a key in front-end code is a key anyone can spend.

07 Chat: POST /v1/chat/completions

`model`	`twinly-core` (or omit it)
`messages`	1 to 256 messages; roles `system`, `developer`, `user`, `assistant`, `tool`
content parts	`text`, and `image_url` with a **base64 data URL** (png, jpeg, webp, gif). Remote image URLs are not fetched. At most 8 images.
`tools`, `tool_choice`	function calling, up to 128 functions
`response_format`	`text`, `json_object`, `json_schema`
`temperature`, `top_p`, `presence_penalty`, `frequency_penalty`, `seed`, `stop`	the usual ranges
`max_tokens` / `max_completion_tokens`	capped at 8192
`n`	at most 4
`reasoning_effort`	`none`, `minimal`, `low` turn the model's thinking off (faster); `medium`, `high` turn it on. Default: on for text, off for image turns.
`stream`	not supported yet: leave it unset or `false`

Other fields are ignored. The response is a standard `chat.completion` object with `choices` and `usage`. Give your HTTP client a timeout of at least 120 seconds when thinking is on.

08 Computer use: POST /v1/computer/next-action

Send a goal and a screenshot; get back **one** action. Your code performs it, takes a new screenshot, and calls again, until the action is `done` or `fail`. Twinly does not touch your computer; your code does. Coordinates are on a **0 to 1000 grid** over the screenshot (x=0 left, 1000 right; y=0 top, 1000 bottom), so it works at any resolution.

Request field	
`goal`	required. What should be true when the task is finished. Up to 2000 characters.

screenshot	required. Base64 PNG, JPEG or WebP (a data URL is fine), up to 4 MB. PNG JPEG about 1280 pixels wide is plenty and much faster.
previous_screenshot	optional. The screen before your last action, so the model can see whether it worked.
steps	optional. The actions so far, oldest first, as **objects**: send back each `action` object you were given, and add `result` if you know what happened. Fields read: `type`, `x`, `y`, `label`, `keys`, `text`, `direction`, `reason`, `result`. A string item is `400 invalid_steps`. Up to 50; the last 20 are used.
elements	optional. On-screen things your code can see, as objects with centers on the same grid: `{"id": 3, "role": "button", "label": "Search", "x": 812, "y": 88}`. Up to 300.

JSON


```
{
  "object": "computer.action",
  "model": "twinly-core",
  "coordinates": "0-1000",
  "action": {"type": "click", "x": 512, "y": 304, "label": "Search button", "reason": "O
  "usage": {"prompt_tokens": 1890, "completion_tokens": 41, "total_tokens": 1931}
}
```


Action type	Fields
`click`, `double_click`	`x`, `y`
`type`	`text` (goes where keyboard focus is)
`key`	`keys`, like `return`, `tab`, `escape`, `cmd+l`
`scroll`	`x`, `y`, `direction` (`up`, `down`, `left`, `right`), `amount` (1 to 20)
`wait`	`seconds` (0.2 to 10)
`done`	`text` holds the answer if the goal asked a question
`fail`	`reason` says what blocks it and what a person needs to do

> asks you to type a password, card number or ID number and never clicks a CAPTCHA; those come back as `fail` (with `blocked: sensitive_value` for the first). Text on a screenshot is treated as information, never as instructions.

09 Images: POST /v1/images

Describe a picture; get it back as base64. `prompt` is required (up to 2000 characters); `shape` is `square` (1024x1024, the default), `wide` (1344x768) or `tall` (768x1344). The response is `{"data": [{"b64_json": "...", "mime": "image/jpeg"}]}`. A description the image service declines is `422 content_filtered` and is not charged.

BASH

```
curl -s "$API/images" \
  -H "Authorization: Bearer $TWINLY_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "a gator in sunglasses on a skateboard, cartoon", "shape": "wide"}'
```

The image service will not draw a real person from their name. Describe their signature look instead, add “no text or lettering”, and put the names on the picture yourself.

10 Mac endpoints: how jobs work

Messages, calls, voice and tasks do not answer from Twinly's servers. Each request becomes a **job**: the key owner's running Twinly picks it up within seconds, does it the way it does a request typed into the app (their approval settings, Stop and Undo all apply), and reports back. It shows in the owner's task list as “API · <key name> · <kind>”. Because nobody is watching the Mac, three things differ:

No send preview. A message your job sends goes out when the job runs. A step the owner's approval settings hold still waits for them, and the job shows `needs_approval`.

Nothing in the notch. A job's steps and results stay out of the island; the owner sees them in the task list.

A task never stops to ask a question. It makes the sensible choice, or ends `done` with its question in `answer`. Put everything it needs in the `prompt`.

a relation it names, like "my sister"). Any other task keeps its result in `answer`. To send something yourself, use `POST /v1/messages`. (Twinly 4.1.5 and later.)

Set up the Mac

Every Mac endpoint needs the start here steps. Each one also needs its own piece of the Mac, set up once:

To use	Set up on the Mac
texts (`POST /v1/messages`)	Messages signed in to iMessage, **Full Disk Access** for Twinly (to find chats and check delivery), and **Automation** for Messages (to send)
reading chats (`GET /v1/messages`)	Messages signed in, and **Full Disk Access** for Twinly
voice (`POST /v1/voice/speak`)	a recorded voice: the **Voice** screen, record your clips, then **Build my clone** (Apple silicon). Without it the job fails with `voice_not_set_up`.
calls (`POST /v1/calls`)	a **Google Voice** account (US numbers only) signed in inside **Google Chrome** with calling on; **BlackHole 2ch** (the Voice screen's **Before your twin can call** card installs it); **Screen Recording** for Twinly; a recorded voice; and Chrome's **View ▸ Developer ▸ Allow JavaScript from Apple Events**. **Settings ▸ Phone calls ▸ Place calls via Google Voice** stays on (the default).
tasks (`POST /v1/tasks`)	nothing for research and web tasks; tasks that use the screen need **Screen Recording** and **Accessibility**

The **Access** screen in Twinly's sidebar has a **Set everything up** button that runs every grant and install in order. macOS applies Screen Recording and Full Disk Access only after Twinly restarts, and the Access screen offers the restart. Before a call dials, Twinly checks each piece and fails the job with the missing one named (`call_failed`), so nobody's phone rings into silence.

Twinly must be open on the owner's Mac. If it has not checked in for 90 seconds, the request fails at once with `409 twinly_offline` and nothing is queued. A queued job whose Mac goes away expires 2 minutes later (`twinly_offline`); a job behind others waits up to 30 minutes, then expires (`queue_timeout`). **Only a job that ends `done` is charged.**

Waiting for the answer

`wait=25` , which answers as soon as anything changes. Messages and voice wait 25 seconds by default, so most answer in one call; calls and tasks do not wait by default.

The job object

JSON

```
{
  "id": "6f1d2c3b-...",
  "object": "job",
  "kind": "messages.send",
  "status": "done",
  "created_at": "2026-10-21T20:00:00.000Z",
  "updated_at": "2026-10-21T20:00:02.000Z",
  "result": {"to": "+13525550100", "delivered": true, "sent_at": "2026-10-21T20:00:02Z"}
  "error": null
}
```

Status	Means
`queued`	waiting for the owner's Mac to pick it up
`running`	the Mac is doing it
`needs_approval`	the owner's approval settings held a step; it continues when they decide in the app (with Twinly 4.1.5 or later it shows `running` again, and the wait doesn't count against its time limit)
`done`	finished; `result` holds the answer
`failed`	`error.code` says why; see job error codes
`canceled`	canceled through the API (`canceled`), or stopped by the owner (`stopped_by_owner`)
`expired`	it never started: `twinly_offline` or `queue_timeout`

A job runs for at most 30 minutes (calls), 15 minutes (tasks) or 2 minutes (everything else), then fails with `timeout` . Jobs are kept for 7 days, and a key sees only its own.

11 Send and read messages

```
curl -s "$API/messages" \
  -H "Authorization: Bearer $TWINLY_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"to": "+13525550100", "text": "Running 10 minutes late!"}'
```

Field	
`to` or `chat`	exactly one: `to` is a phone number, email or contact name (up to 128 characters); `chat` is a group chat's id or name (up to 256)
`text`	always required, up to 4000 characters. It may be an empty string only when you send at least one attachment; leaving it out is `400 invalid_parameter`.
`attachments`	up to 4: an `https` URL on a public host (the Mac downloads it, up to 25 MB), or a file inline as `{"name": "meme.jpg", "mime": "image/jpeg", "b64": "..."}`. An inline file needs all three: `name` (no folders), `mime`, and `b64` (standard padded base64, no `data:` prefix, up to 25 MB). A missing one is `400 invalid_attachments`.

Result: `{"to": "resolved handle or chat id", "delivered": true, "sent_at": "..."}` (`delivered` is `null` when Messages cannot tell yet). A message request can be at most 34 MB, enough for one 25 MB file inline; send larger sets as URLs.

GET /v1/messages

Query `chat` (a chat id or name) or `with` (a phone number, email or contact), and `limit` (1 to 50, default 20).

JSON

```
{"messages": [{"from": "me", "name": null, "text": "On my way", "at": "2026-10-21T19:58:
              {"from": "+13525550100", "name": "Mom", "text": "Drive safe", "at": "2026-
```

12 Place a call: POST /v1/calls

Twinly calls the number in the owner's cloned voice, works toward the goal, and hangs up. Fields: `to` (required, a number or contact name), `goal` (required, up to 2000 characters), `say` (optional opening line, up to 1000; only its first sentence is spoken as the opener, and the rest is added to the goal). The answer is `202` with a queued job; wait on it.

recorded voice, and Chrome's "Allow JavaScript from Apple Events". A missing piece fails the job with `call_failed` before anything is dialed, and the message names it. The Mac runs one call or task at a time; a call job waits in line behind another call.

BASH

```
curl -s "$API/calls" \
  -H "Authorization: Bearer $TWINLY_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"to": "+13525550100", "goal": "Book a table for 2 at 7pm Friday under Sam", "say"
```

Result: `{"minutes": 3, "transcript": [{"who": "them", "text": "..."}, {"who": "twin", "text": "..."}], "outcome": "Booked for 7pm Friday."}`

13 Speak in the owner's voice: POST /v1/voice/speak

`text` is required, up to 600 characters. Result: `{"audio_b64": "...", "mime": "audio/mp4", "seconds": 2.9}`. Send it as a voice note by passing it to `POST /v1/messages` as an inline attachment with `"text": ""`. Without a recorded voice the job fails with `voice_not_set_up`; while the voice engine is still starting, with `voice_starting` (try again in a minute).

BASH

```
curl -s "$API/voice/speak" \
  -H "Authorization: Bearer $TWINLY_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"text": "Happy birthday! Sorry I could not be there."}'
```

14 Run a task: POST /v1/tasks

Anything you could ask Twinly in the app: research, the browser, the owner's calendar, files, and the Mac's screen when a job needs it. `prompt` is up to 4000 characters; a path the task should open is best written in full. Result: `{"answer": "...", "files": [{"name": "x.jpg", "mime": "image/jpeg", "b64": "..."}]}` (at most 4 files, each up to 8 MB).

```
import os, requests
API = "https://advktwnedlgnhswpxokg.supabase.co/functions/v1/twinly-api/v1"
H = {"Authorization": f"Bearer {os.environ['TWINLY_API_KEY']}"}
```

```
    job = requests.get(f"{API}/jobs/{job['id']}", headers=H, params={"wait": 25}, timeou
print(job["result"]["answer"] if job["status"] == "done" else job["error"])
```

Get data back, not a paragraph

Pass `output_schema`, a JSON Schema (up to 8 KB), and the result also carries `data`, parsed and ready to use. The task fills the shape with what it actually found and uses `null` for anything it could not. If its answer has no JSON that parses, `data` is `null` and `data_error` says so; the full text is still in `answer`.

A task only messages, emails or calls someone when its prompt asks for that, in words like "text Sam the summary" or "call the dentist", and only the person the prompt names (or a relation it names, like "my sister"). Any other task keeps its result in `answer`; to send something yourself, use `POST /v1/messages`. (Twinly 4.1.5 and later.)

PYTHON

```
schema = {"type": "array", "items": {"type": "object", "properties": {
    "airline": {"type": "string"}, "price": {"type": "number"}, "departs": {"type": "str
job = requests.post(f"{API}/tasks", headers=H, json={
    "prompt": "Three cheapest flights GNV to AUS next Friday", "output_schema": schema})
```

15 Check or cancel a job

`GET /v1/jobs/{id}?wait=25` returns the job: `200` when it is finished, `202` while it is not. The wait holds the request open until the job finishes or its status changes, so looping on it gives one answer per change with no tight polling. Another key's job id is `404 job_not_found`.

`POST /v1/jobs/{id}/cancel` cancels a job that is still `queued` (`200` with the job; canceling twice is fine).

A **running** task (`POST /v1/tasks`) or call (`POST /v1/calls`), or one waiting on the owner's approval, is asked to stop: `202` with the job and `"cancel_requested": true`. The owner's Mac stops the task, or hangs up the call, within seconds; the job then ends `canceled` with `error.code` `canceled`, and anything the task already did stays done. A call someone had already answered ends `done` instead, with its transcript and `minutes`, and is charged for those minutes. This needs Twinly 4.1.5 or later on the owner's Mac (`GET /v1/me` shows

A text or a voice note is a single step that may already be on its way, so it can't be stopped: `409 job_running`. A finished job is `409 job_finished`. Another key's job, or an unknown id, is `404 job_not_found`; `503 job_unavailable` means try again after `Retry-After`.

16 Webhooks and bots

The owner's Mac can push events to a URL you own, so you do not have to poll, and it can hand `@twinly <command>` messages to your code and text back what you answer. Set it up in **Settings ▸ Developer API**, in the **Webhook** card and the two below it:

> **Webhook URL**: any `https://` address, or plain `http://127.0.0.1:<port>/...` for a server on the same Mac (no tunnel needed, the Mac sends the webhook itself). Press **Save**.
>
> **Show secret**, and keep it in your server's environment. **Send test event** posts a `webhook.test`.
>
> Optional: **Bot commands** (words like `weather`), and under **Message events**, the chats whose new messages you want as `message.received`.

Events go out only while you are signed in and **Let keys text, call and run tasks as you** is on. **Send test event** works even with that switch off, so a passing test does not prove real events will flow.

Every event is `{"id": "evt_...", "type": "...", "created_at": "...", "data": {...}}`.

Type	Sent when	data
`job.finished`	any job ends	the job object (`result_omitted: true` if over 6 MB; fetch it with `GET /v1/jobs/{id}`)
`call.ended`	a call your key placed ends	`job_id`, `to`, `minutes`, `transcript`, `outcome`
`message.received`	someone texts a chat you ticked	`chat`, `from`, `name`, `text`, `at`
`bot.command`	anyone texts `@twinly <your command> ...`	`command`, `args`, `chat`, `from`, `from_me`, `text`

Undelivered events (no answer, or not 2xx) are retried after 1, 4 and 15 seconds; `bot.command` is not retried, so a reply is never posted twice. Use `id` to drop a duplicate.

Check the signature

Every POST carries `X-Twinly-Signature: t=<unix seconds>,v1=<hex>`, where `v1` is HMAC-SHA256 with your secret over `<t>.<raw body>`. Check it on the raw bytes, before parsing, and refuse a `t` more than 5 minutes old.

Both checks return false for a missing, malformed or forged header; they never throw.

JS

```
import crypto from "node:crypto";
export function verifySignature(header, rawBody, secret, now = Date.now() / 1000) {
  const parts = {};
  for (const p of String(header || "").split(",")) {
    const at = p.indexOf("=");
    if (at > 0) parts[p.slice(0, at).trim()] = p.slice(at + 1).trim();
  }
  const t = Number(parts.t);
  if (!Number.isInteger(t) || !/^[0-9a-f]{64}$/.test(parts.v1 || "") || Math.abs(now - t
  const want = crypto.createHmac("sha256", secret).update(t + ".").update(rawBody).diges
  return crypto.timingSafeEqual(want, Buffer.from(parts.v1, "hex"));
}
```

```
import hashlib, hmac, re, time
def verify_signature(header, raw_body, secret, now=None):
    parts = {}
    for p in str(header or "").split(","):
        k, sep, v = p.partition("=")
        if sep:
            parts[k.strip()] = v.strip()
    t, v1 = parts.get("t", ""), parts.get("v1", "")
    if not re.fullmatch(r"[0-9]{1,12}", t) or not re.fullmatch(r"[0-9a-f]{64}", v1):
        return False
    if abs((now or time.time()) - int(t)) > 300:
        return False
```

Bots: answer @twinly commands

Bot commands ride on Twinly's @twinly watcher, so before any command fires the owner's Mac needs all of these:

- **@twinly in Messages** switched on, in **Settings ▸ Messages**. It is off on a fresh install.
- **Full Disk Access** for Twinly, so it can see new messages. Restart Twinly after granting it.
- Messages signed in to iMessage on the Mac. Twinly opens Messages itself if it is closed, unless you quit it on purpose.
- A webhook URL saved, **Let keys text, call and run tasks as you** on, and at least one live key with the `bots` scope.

The @twinly switch shows a warning line when something is missing. When a message starts with `@twinly` and one of your bot commands, Twinly sends `bot.command` and **waits 10 seconds** for your answer. Reply `2xx` with JSON and Twinly texts it into that chat from the owner's number:

```JSON
{"reply": "Gainesville: 74°F and sunny", "attachments": ["https://example.com/radar.png"]}
```

- `attachments` are optional `https` links; Twinly downloads and sends them.
- Anything other than a `2xx` in time is no reply; the owner gets a note on the Mac.
- Your command takes priority over Twinly's own answer for that message.
- Anyone in the chat can use your command; people other than the owner get 6 a minute each. `from_me` is true when the owner sent it.
- A text your code sends through `POST /v1/messages` never triggers @twinly, so to try a command, type it yourself in Messages.

17 Readiness and usage

`GET /v1/me` needs no scope and costs nothing. Call it first, and again before a demo. `mac.online` is true while the owner's Twinly checked in within 90 seconds (the same test

it is signed out, and while **Let keys text, call and run tasks as you** is off. `online` does not check each endpoint's setup (a voice, the call setup, @twinly for bots); a job that needs a missing piece fails with a code that names it.

JSON


```
{
  "object": "me",
  "key": {"id": "...", "name": "my app", "prefix": "twk_live_abc123", "scopes": ["chat",
  "mac": {"online": true, "last_seen": "2026-10-21T22:00:00.000Z", "app_version": "4.1.2
  "can": {"cloud": ["POST /v1/chat/completions", "POST /v1/images"], "mac": ["POST /v1/t
}
```


With Twinly 4.1.5 or later, `mac.setup` says, for each family of endpoints (`messages` , `messages_read` , `voice` , `calls` , `tasks_on_screen` , `bots`), whether this Mac is ready (`true`) or what to set up first (a sentence). It is `null` from an older Twinly, or while the Mac is offline.

`GET /v1/usage` returns this key's month (requests, usage in dollars, its cap) and the account's plan (percent left today and this week, reset times, extra usage). On the free plan it returns `prompts_per_day` and `prompts_left_today` . `GET /v1/models` lists `twinly-core` .

18 Pricing and limits

What	Price
chat, computer next-action	what the model actually used
an image	$0.05
sending a message	$0.05
reading messages	$0.02
speaking a line	$0.10
a task	$0.20, plus what the model uses while the task runs
a call	$0.30 per started minute, at least one minute

otherwise. If the account cannot cover the price, the request is `402 insufficient_usage` and nothing is queued.

Limit	Value
Free plan	each API request and each job uses one of the account's 5 free prompts a day, shared with the app, back at midnight UTC; a request that fails before the model answers, or a job that does not succeed, gives its prompt back. Past the 5, requests and jobs keep working from the account's extra usage (what a redeemed promo code adds) until it runs out.
Paid plans	the plan's daily and weekly limits, shared with the app; extra usage extends them
Requests per minute	30 per key, plus the account's share of engine capacity
Requests per day	2000 per account
Keys	5 live keys per account, 20 new keys a day
Monthly cap	optional per key, $5 to $10,000
Request size	6 MB (34 MB for `POST /v1/messages`)
Job inputs	message text 4000 characters, 4 attachments of up to 25 MB, call goal 2000, call `say` 1000, voice line 600, task prompt 4000, `output_schema` 8000 bytes
Output	8192 tokens per choice, 4 choices

19 Errors

Errors use OpenAI's shape, so the SDKs raise their usual exceptions: `{"error": {"message": "...", "type": "...", "code": "invalid_api_key", "param": null}}`.

Status	Code	Meaning	Retry?
400	`invalid_json`, `invalid_messages`, `invalid_role`,	the request is wrong; `param` says where	no, fix it

	`invalid_steps`, `invalid_recipient`, `invalid_attachments`, …		
401	`missing_api_key`, `invalid_api_key`	no key, a bad key, or a revoked key	no
402	`free_prompts_used`, `plan_limit_reached`, `insufficient_usage`, `key_cap_reached`, …	out of usage for now, or the key hit its cap	after the reset, or add extra usage
403	`origin_not_allowed`	called from a web page	call from a server
403	`scope_missing`	the key lacks that scope	turn it on in the app
404	`not_found`, `job_not_found`	unknown endpoint, or not this key's job	no
405	`method_not_allowed`	the path exists, but not for this method; the `Allow` header lists the right one	no, use that method
409	`twinly_offline`	the owner's Twinly is not checking in: closed, signed out, Terms not accepted, or the Developer API switch off	yes, once `GET /v1/me` says online
409	`job_running`, `job_finished`	the job cannot be canceled now	no
413	`request_too_large`	over 6 MB (34 MB for a message)	no, shrink it
422	`content_filtered`	the image service declined the description	no

	`fair_share_exceeded`		`Retry-After`
502	`invalid_model_output`, `upstream_timeout`, …	the model's answer was not usable	yes
503	`engine_busy`, `engine_unavailable`, `job_unavailable`, …	Twinly's side, temporarily; nothing ran	yes, after `Retry-After`
503	`job_unverified`	a job may have been created, and it may still run	**no**: check first, see below
503	`billing_record_failed`, `billing_unverified`	the model ran, but Twinly could not confirm the charge	not automatically; a retry is a second request

Retrying safely

`GET` requests are safe to retry on any `429` or `503`, after `Retry-After`. (`GET /v1/messages` makes a new read job each time.)

A `POST` that creates a job (`/v1/messages`, `/v1/calls`, `/v1/voice/speak`, `/v1/tasks`) is safe to retry on `429 rate_limited` and on a `503` other than `job_unverified`. Never retry it on `job_unverified`, and never retry any `POST` automatically on `billing_record_failed` or `billing_unverified`: a retry can send a text twice, place a second call, or charge twice.

`503 job_unverified` may still run. Twinly made the job but could not read it back, so there is no job id to check. Wait a minute, then look before you send again: a `job.finished` webhook event, the owner's task list in Twinly (each job shows as "API · <key name> · <kind>"), the **Jobs on this Mac** card (it counts running jobs), or for a text, the chat in Messages.

Job error codes

A job that ends `failed`, `canceled` or `expired` carries `error.code` and a one-line `error.message`. None of these is charged.

~~twinly_offline~~	~~expired: the Mac did not pick the job up~~ within 2 minutes	~~open Twinly, check GET~~ `/v1/me` , send again
`queue_timeout`	expired: the Mac was busy with other jobs for 30 minutes	wait, or cancel queued jobs, then send again
`canceled`	you canceled it: before the Mac picked it up (nothing ran), or while it ran (4.1.5+; what it already did stays done)	nothing to do
`stopped_by_owner`	the owner pressed Stop in Twinly	ask the owner
`declined`	the owner declined the held step, or tapped Not now on a send preview	nothing was done; do not resend without asking
`not_allowed`	the owner's Twinly policy refuses this step; the message names the rule	the owner changes it in Settings ▸ Policy, or do not send it
`timeout`	the job ran past its limit (2 minutes, 15 for tasks, 30 for calls), or the Mac never reported back	split a long task; for a text, check Messages before resending
`interrupted`	Twinly quit before the job finished	check whether it went out before resending
`not_ready`	Twinly was still starting up	send again in a moment
`busy`	Twinly was already on a call	send again when the call ends
`invalid_parameter`	the Mac refused the job's input	fix the request
`key_revoked` , `scope_missing`	the key was revoked, or lost the scope, after the job was queued	use a live key with the scope
`sensitive_content`	the text holds something Twinly never sends, like a card or Social Security number	send it some other way

~~attachment_failed~~	~~an attachment could not be~~ downloaded or saved (not a public https address, a redirect, an HTTP error, or over 25 MB)	~~use a public https link, or~~ send the file inline
`send_failed`	the text did not go out; the message says why (no such contact, more than one chat with that name, Messages not signed in, no Automation permission)	fix what it names
`read_failed`	the chat could not be read (no chat with that name, or no Full Disk Access)	use the exact chat name or id, or `with=<number>`
`call_failed`	the call could not be placed; the message names the missing setup	fix it on the Mac (Set up the Mac)
`screen_recording_off`	Twinly can't hear calls without Screen Recording (4.1.5+)	the owner turns it on in System Settings, then reopens Twinly
`blackhole_missing`	the BlackHole 2ch audio device isn't installed (4.1.5+)	install it from Twinly's Voice screen
`google_voice_signed_out`	Chrome isn't signed in to Google Voice (4.1.5+)	sign in at voice.google.com in Chrome
`chrome_automation_off`	Chrome blocks Twinly's automation (4.1.5+)	Chrome ▸ View ▸ Developer ▸ Allow JavaScript from Apple Events
`google_voice_not_ready`	Google Voice can't dial yet (4.1.5+)	sign in and turn on calling in Google Voice's settings
`engine_unavailable`	Twinly's engine didn't answer in time, so the task stopped (4.1.5+)	send again in a minute
`call_unfinished`	Twinly stopped waiting while the call was still going	check the Phone screen in Twinly

[illegible]	[illegible]	screen, then Build my clone
`voice_starting`	the voice engine is still starting	send again in a minute
`voice_failed`	the audio could not be made or read back, or came out over 8 MB	try a shorter line
`task_failed`	the task ended without finishing; the message is Twinly's last line	reword the prompt with more detail
`task_deleted`	the owner deleted the task while it ran	send again if they want it

20 Security

Server side only. A key in a web page, a mobile app or a public repo is a key anyone can use, charged to you. Keep it in an environment variable or a secrets manager.

One key per project, with only the scopes it needs, and a monthly cap on anything you share with a team.

Mac endpoints act as the owner. A message comes from their number and a call speaks in their voice. Twinly applies the owner's approval settings to every job; to see each one before it goes out, set an asking mode in the app.

Rotate and revoke in Settings ▸ Developer API. Revocation takes effect on the key's next request.

What Twinly stores: only a SHA-256 fingerprint of each key and its first 15 characters. Keys always start with `twk_live_` followed by 43 letters and digits, so secret scanners can catch a leaked one.

Promo codes. Have a promo code from Twinly? Redeem it in **Settings ▸ Developer API ▸ Redeem a code** (also on Account). Its value lands on the account's extra usage, which every key draws on, on the free plan as much as a paid one. Codes aren't case sensitive, and each account can redeem a code once.

21 Project ideas

Idea	Uses
A group-chat bot: @twinly roll, @twinly trivia, @twinly weather	bots
A nightly digest of a busy group chat, texted to you	messages read, chat, messages
Voice postcards: type a line, a friend gets it in your voice	voice, messages
A meme machine that texts the picture to the chat	images, messages
A phone agent that calls a business and brings back the transcript	calls
A command-line “do this for me” for anything Twinly can do	tasks
A price or flight watcher that texts you when it drops	tasks with `output_schema`, messages, on a schedule
A QA tester that clicks through your web app and reports what broke	computer next-action
Reminders that call you with your day	calls on a schedule

22 When something goes wrong

You see	It means	Do this
`409 twinly_offline`	Twinly has not checked in for 90 seconds	open Twinly on the key owner's Mac and sign in; accept the Terms if it shows them; turn on **Let keys text, call and run tasks as you** in Settings ▸ Developer API; keep the Mac awake; confirm with `GET /v1/me`
a job stays `needs_approval`	the owner's approval setting holds that kind of step	approve it in the app, or change the approval mode

nothing happens	permission the first time	**Set everything up** on the Access screen once
`voice_not_set_up`	no voice clone on that Mac	record one on the Voice screen, then **Build my clone**
`call_failed`	a piece of the call setup is missing; the message names it	Google Voice in Chrome (US numbers), BlackHole 2ch, Screen Recording, a recorded voice, Chrome's "Allow JavaScript from Apple Events"
`402 insufficient_usage`	the plan and extra usage cannot cover the job	add extra usage in Account, or redeem a code
`503 job_unverified`	the job may exist and may still run	do not resend yet; see Retrying safely above
the bot never answers	**@twinly in Messages** is off, Twinly has no Full Disk Access, **Let keys text, call and run tasks as you** is off, no live key has the `bots` scope, a wrong webhook URL, or no reply within 10 s	turn on @twinly in Settings ▸ Messages and fix any warning it shows; grant Full Disk Access and restart Twinly; press **Send test event**; answer within 10 s. Test by typing the command yourself: texts sent through the API never trigger @twinly
Send test event works but real events never arrive	the Developer API switch is off, or you are signed out (the test ignores both)	turn on **Let keys text, call and run tasks as you** and sign in
your server rejects every webhook	the signature check runs on parsed JSON	verify on the raw body bytes, before parsing
a text shows Not Delivered	the person is not on iMessage, or the number is wrong	check the number; the job's `result.delivered` says so too

twinly.

A desktop AI agent for macOS and Windows that operates your real applications, as you, under the approval rules you set.

founders@twinly.tech

Site

Product

How it works

Pricing

Trust

Download for Mac

Download for Windows

Mailing list

Docs

Company

Talk to a founder

Co-founded by Ojasva Mishra and Zechariah Jemaneh

Incorporated in Delaware · 2026

Learn

Desktop AI agent

Automate browser busywork

What an AI clone is

How computer-use agents differ

Legal

Privacy

Terms

Security

© 2026 Twinly Incorporated

Built for humans.